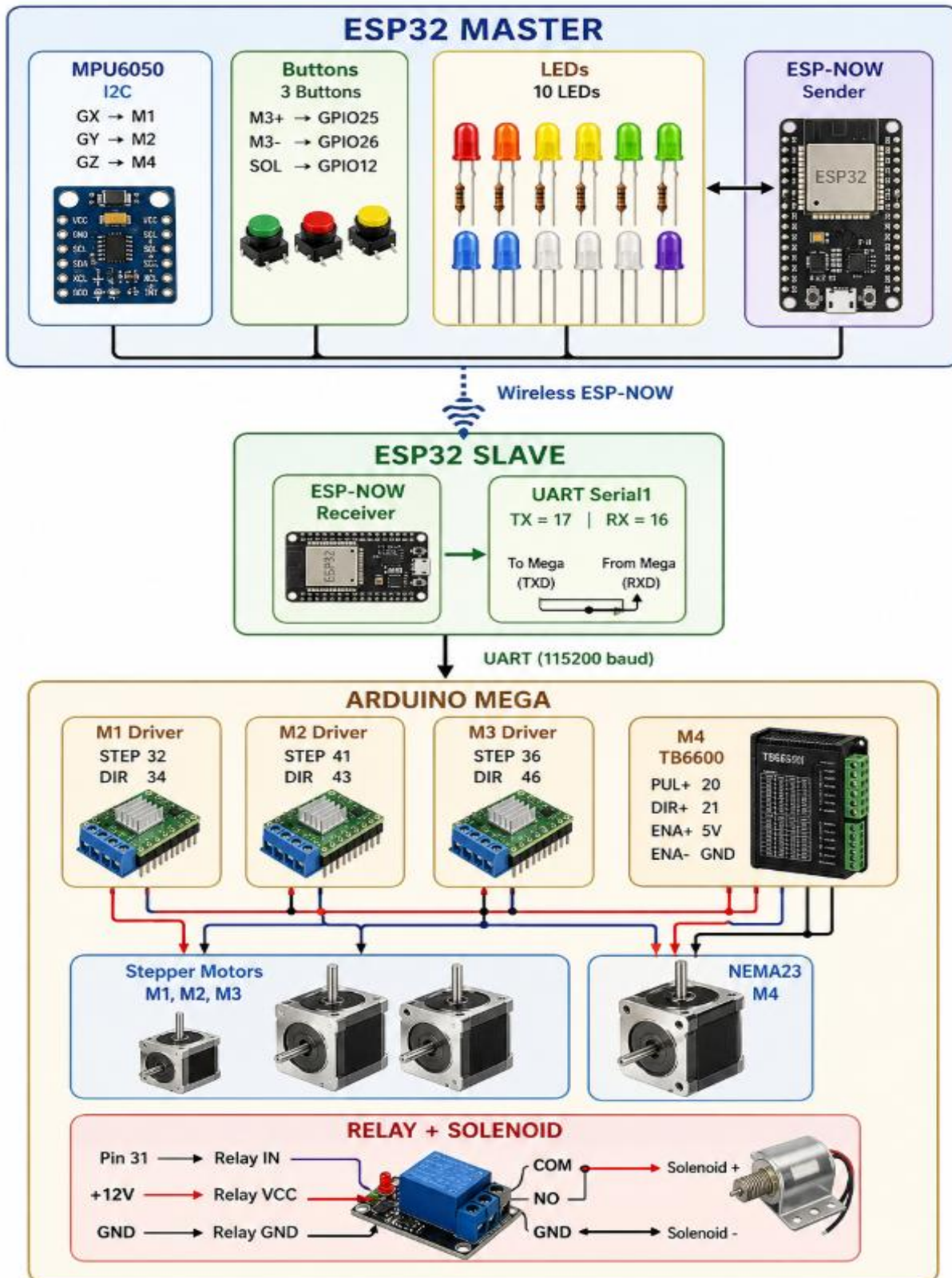
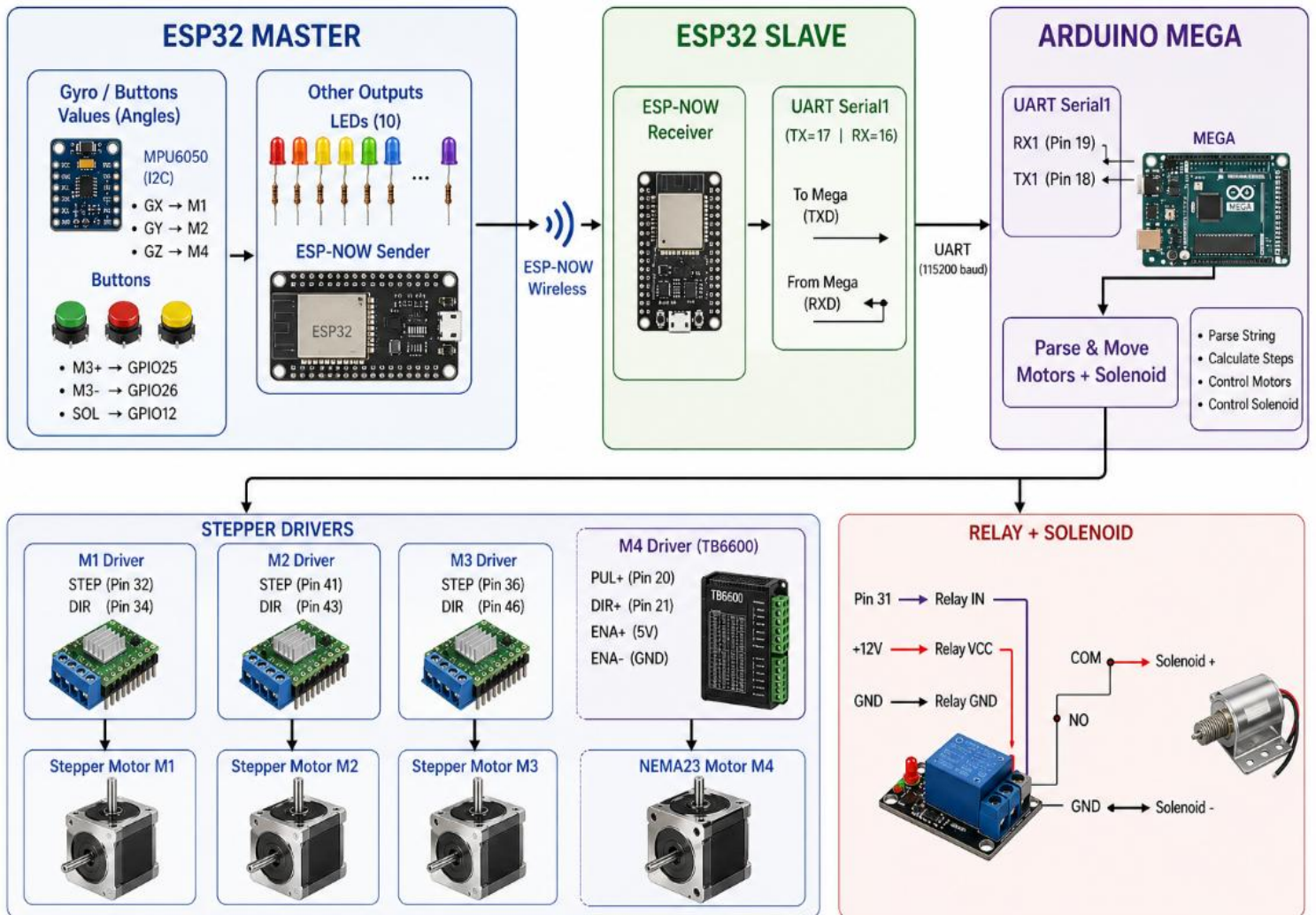
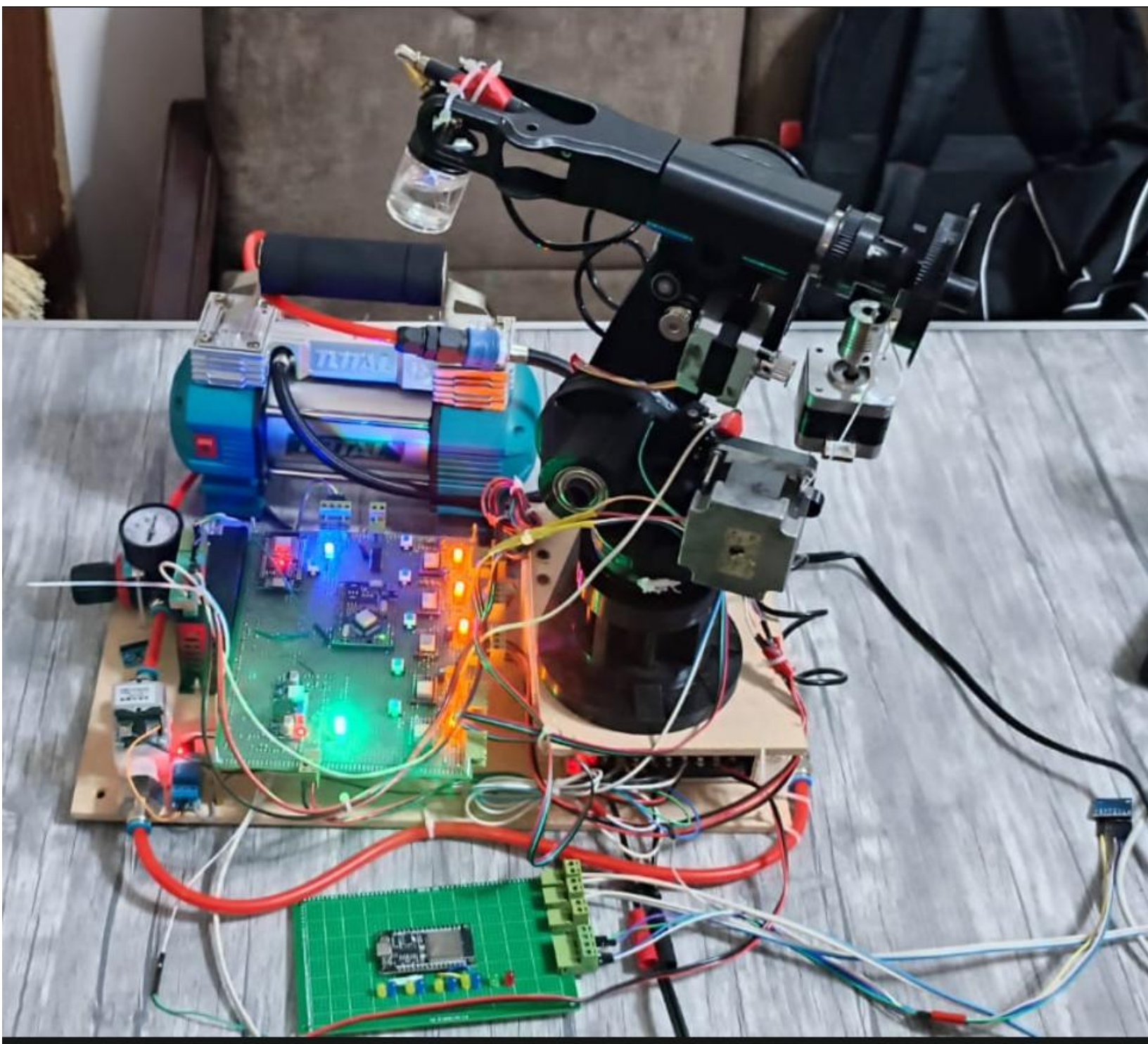


Diagram



Data Flow:





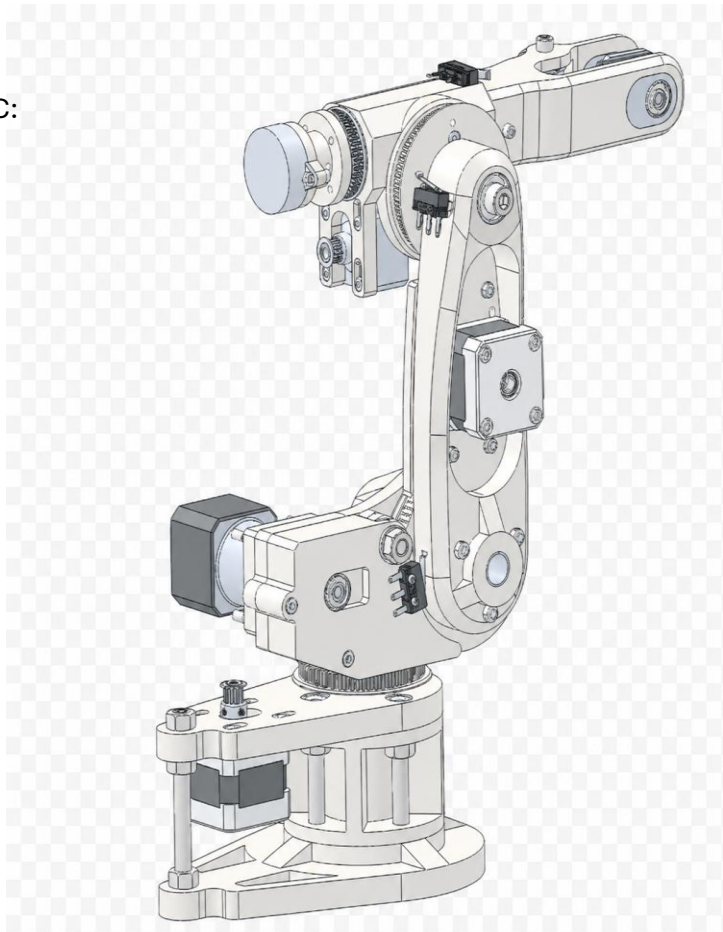
Schematic Drawing):

Create a detailed electronic schematic diagram for a robotic arm control system using Proteus 8 Professional style.

SYSTEM COMPONENTS:

1. ESP32 MASTER Module (Top Left)

- Label: "ESP32 MASTER"
- MPU6050 Gyroscope connected via I2C:
 - * VCC → 3.3V
 - * GND → GND
 - * SDA → GPIO21
 - * SCL → GPIO22
 - * AD0 → GND (Address 0x68)
- 3 Push Buttons (INPUT_PULLUP):
 - * M3+ Button → GPIO25 + GND
 - * M3- Button → GPIO26 + GND
 - * SOLENOID Button → GPIO12 + GND
- 10 LEDs with 220Ω resistors:
 - * M1+ LED → GPIO32
 - * M1- LED → GPIO33
 - * M2+ LED → GPIO19
 - * M2- LED → GPIO18
 - * M4+ LED → GPIO5
 - * M4- LED → GPIO17
 - * M3+ LED → GPIO16
 - * M3- LED → GPIO4



* SOLENOID LED → GPIO13

* GYRO LED → GPIO27

2. ESP32 SLAVE Module (Top Right)

- Label: "ESP32 SLAVE"

- UART Connection to Arduino Mega:

* TX (GPIO17) → Mega RX1 (Pin 19)

* RX (GPIO16) ← Mega TX1 (Pin 18)

* GND → GND

3. Arduino Mega 2560 (Center)

- Label: "Arduino Mega"

- Serial1 Communication:

* RX1 (Pin 19) ← ESP32 TX

* TX1 (Pin 18) → ESP32 RX

- Motor Outputs:

* M1: STEP Pin32, DIR Pin34

* M2: STEP Pin41, DIR Pin43

* M3: STEP Pin36, DIR Pin46

* M4: PUL+ Pin20, DIR+ Pin21

4. Motor Drivers (Bottom Left)

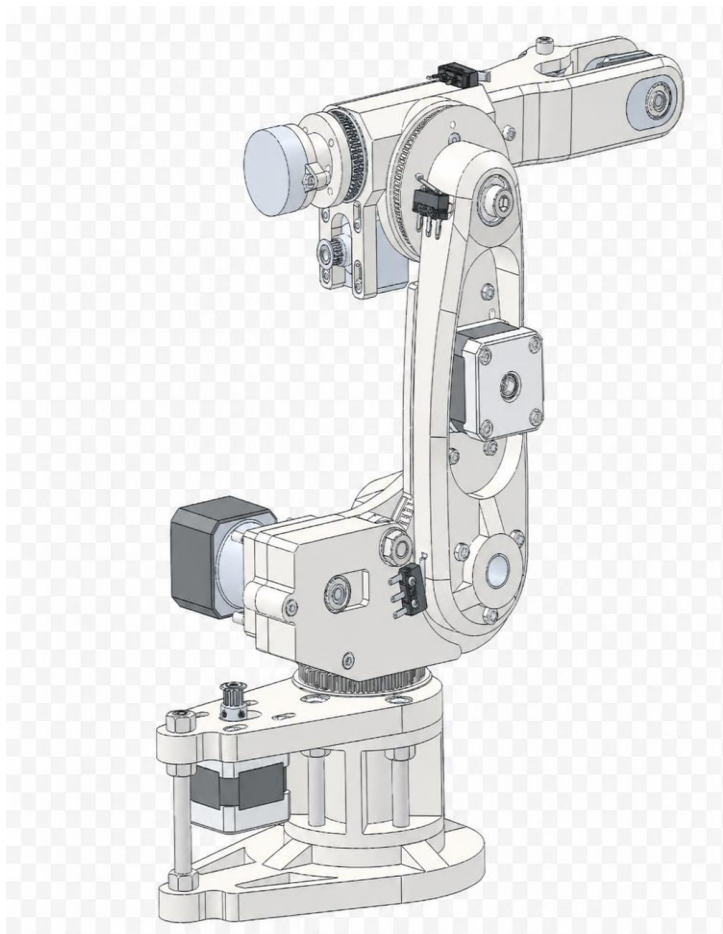
- 3x A4988/DRV8825 Drivers for M1,M2,M3

* STEP, DIR from Mega

* VMOT → +12V

* GND → GND

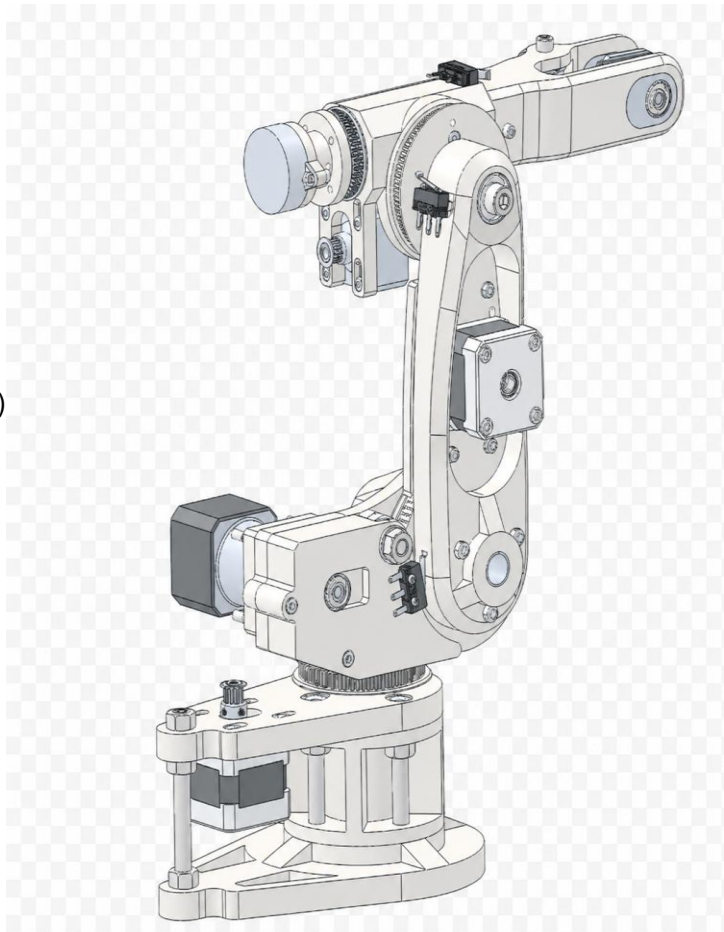
* A1,A2 → Motor Coil 1



- * B1,B2 → Motor Coil 2
- 1x TB6600 Driver for M4 (NEMA23)
- * PUL+ → Mega Pin20
- * PUL- → GND
- * DIR+ → Mega Pin21
- * DIR- → GND
- * ENA+ → 5V
- * ENA- → GND
- * A+,A- → NEMA23 Coil 1
- * B+,B- → NEMA23 Coil 2
- * VCC → +12V
- * GND → GND

5. Relay Module + Solenoid (Bottom Right)

- 12VDC 1-Channel Relay Module
- * VCC → +12V (External Supply)
- * GND → GND (Common)
- * IN → Mega Pin31
- Solenoid Valve
- * COM (Relay) → +12V
- * NO (Relay) → Solenoid+
- * Solenoid- → GND



6. Power Supply Section (Bottom)

- +12V Supply for Motors, Relay, Solenoid
- +5V from Arduino for Logic
- +3.3V from ESP32 for MPU6050

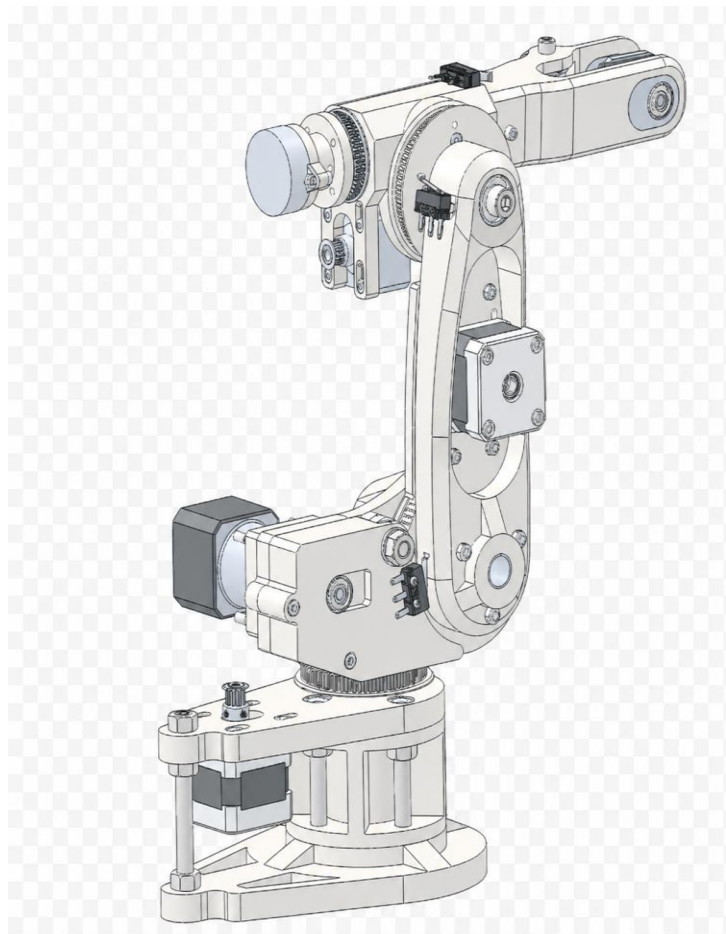
- Common GND for all

WIRELESS CONNECTION:

- Draw dashed line between ESP32 MASTER and ESP32 SLAVE
- Label: "ESP-NOW Protocol (No Router)"

STYLE REQUIREMENTS:

- Clean professional schematic layout
- Standard electronic symbols
- Clear labels and pin numbers
- Color coding:
 - * Red: +12V Power
 - * Orange: +5V Logic
 - * Yellow: +3.3V
 - * Black: GND
 - * Blue: Signal/Data lines
 - * Green: I2C Bus
- Grid layout, left-to-right signal flow
- Title block at bottom right
- Component list table at bottom left



Add annotations:

- "M1,M2,M4: Controlled by Gyroscope"
- "M3: Controlled by Push Buttons ($\pm 90^\circ$)"
- "Solenoid: Momentary Push Button"
- "M4 Limits: 0-720° (2 Rotations)"

Arduino Mega:

```
/*
```

```
 * Arduino Mega - Gyro M1,M2,M4 + Buttons M3 + Momentary Solenoid
```

```
 * FIXED: actualAngleM4 added
```

```
*/
```

```
#include <AccelStepper.h>
```

```
#define SerialMega Serial1
```

```
// Motors
```

```
#define M1_STEP 32
```

```
#define M1_DIR 34
```

```
#define M2_STEP 41
```

```
#define M2_DIR 43
```

```
#define M3_STEP 36
```

```
#define M3_DIR 46
```

```
#define M4_PUL 20
```

```
#define M4_DIR 21
```

```
// Relay
```

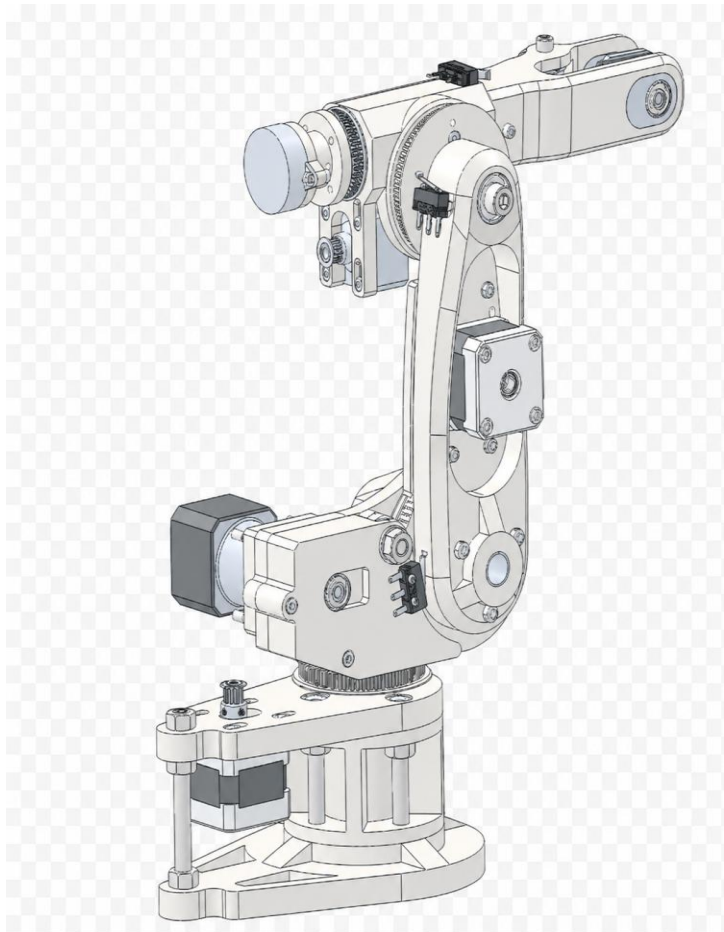
```
#define RELAY_PIN 31
```

```
AccelStepper stepperM1(AccelStepper::DRIVER, M1_STEP, M1_DIR);
```

```
AccelStepper stepperM2(AccelStepper::DRIVER, M2_STEP, M2_DIR);
```

```
AccelStepper stepperM3(AccelStepper::DRIVER, M3_STEP, M3_DIR);
```

```
AccelStepper stepperM4(AccelStepper::DRIVER, M4_PUL, M4_DIR);
```




```
#define STEPS_PER_DEG_M1 10
```

```
#define STEPS_PER_DEG_M2 10
```

```
#define STEPS_PER_DEG_M3 10
```

```
#define STEPS_PER_DEG_M4 20
```

```
#define M3_MIN -90
```

```
#define M3_MAX 90
```

```
#define M4_MIN 0
```

```
#define M4_MAX 720
```

```
#define MOVE_STEPS_M3 50
```

```
int currentAngleM1 = 0, currentAngleM2 = 0, currentAngleM3 = 0, currentAngleM4 = 0;
```

```
bool solenoidState = false;
```

```
int prevAngleM1 = 0, prevAngleM2 = 0, prevAngleM3 = 0, prevAngleM4 = 0;
```

```
int actualAngleM3 = 0;
```

```
int actualAngleM4 = 0; // ← FIXED! Added here
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  SerialMega.begin(115200);
```

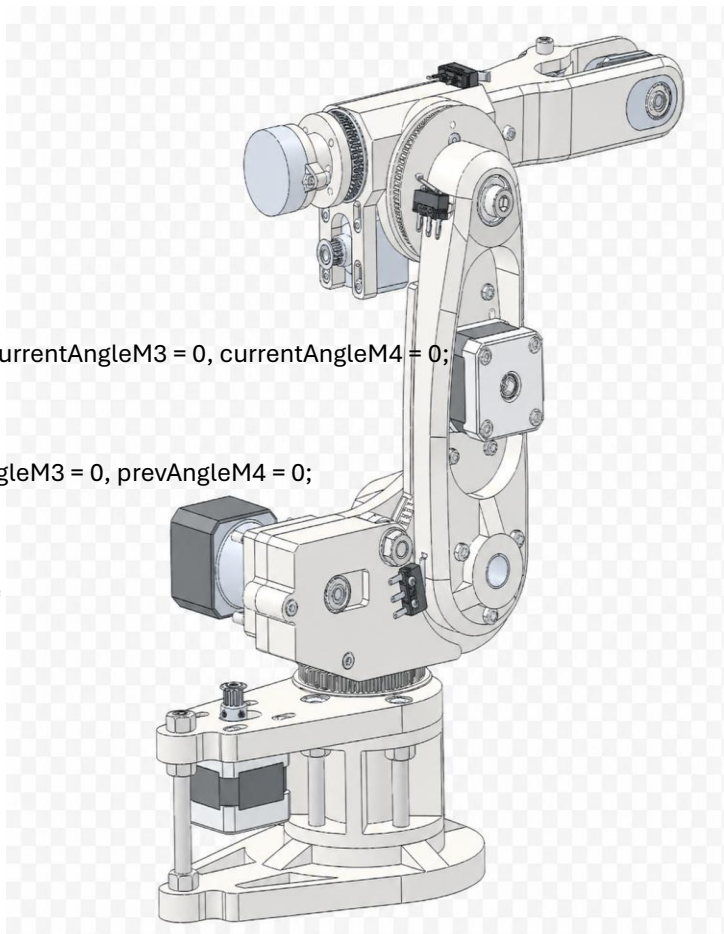
```
  pinMode(RELAY_PIN, OUTPUT);
```

```
  digitalWrite(RELAY_PIN, HIGH); // Active LOW: OFF
```

```
  stepperM1.setMaxSpeed(1000); stepperM1.setAcceleration(500);
```

```
  stepperM2.setMaxSpeed(1000); stepperM2.setAcceleration(500);
```

```
  stepperM3.setMaxSpeed(800); stepperM3.setAcceleration(400);
```



```
stepperM4.setMaxSpeed(1200); stepperM4.setAcceleration(600);
```

```
stepperM1.setCurrentPosition(0);
```

```
stepperM2.setCurrentPosition(0);
```

```
stepperM3.setCurrentPosition(0);
```

```
stepperM4.setCurrentPosition(0);
```

```
Serial.println("MEGA Ready - FIXED");
```

```
}
```

```
void parseCommand(String cmd) {
```

```
    if (!cmd.startsWith("<") || !cmd.endsWith(">")) return;
```

```
    cmd = cmd.substring(1, cmd.length() - 1);
```

```
    int values[5];
```

```
    int idx = 0, lastComma = -1;
```

```
    for (int i = 0; i < cmd.length() && idx < 5; i++) {
```

```
        if (cmd.charAt(i) == ',') {
```

```
            values[idx++] = cmd.substring(lastComma + 1, i).toInt();
```

```
            lastComma = i;
```

```
        }
```

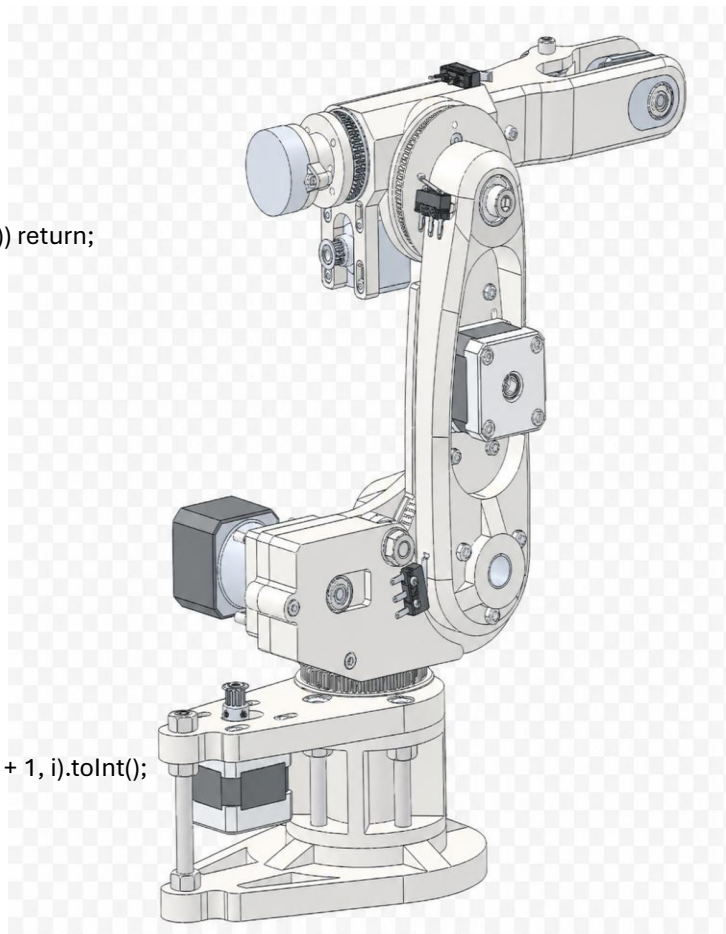
```
    }
```

```
    if (idx < 5) values[idx++] = cmd.substring(lastComma + 1).toInt();
```

```
    if (idx < 5) return;
```

```
    prevAngleM1 = currentAngleM1;
```

```
    prevAngleM2 = currentAngleM2;
```



```
prevAngleM3 = currentAngleM3;  
prevAngleM4 = currentAngleM4;
```

```
currentAngleM1 = values[0];  
currentAngleM2 = values[1];  
currentAngleM3 = values[2];  
currentAngleM4 = values[3];  
solenoidState = (values[4] == 1);
```

```
// M1
```

```
long deltaM1 = (long)(currentAngleM1 - prevAngleM1) * STEPS_PER_DEG_M1;  
stepperM1.move(deltaM1);
```

```
// M2
```

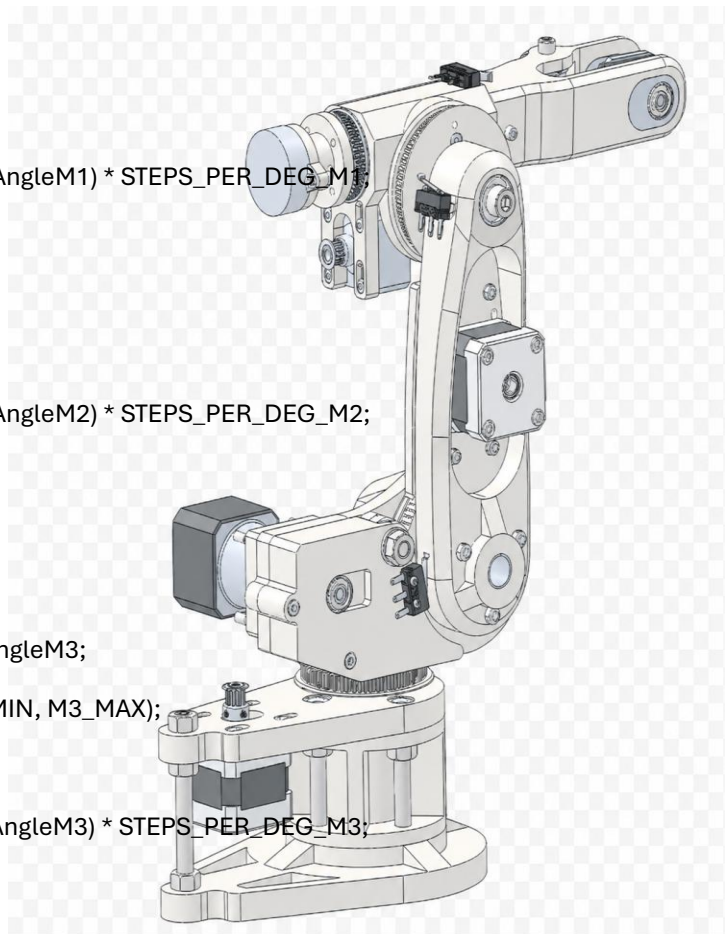
```
long deltaM2 = (long)(currentAngleM2 - prevAngleM2) * STEPS_PER_DEG_M2;  
stepperM2.move(deltaM2);
```

```
// M3 (Buttons  $\pm 90^\circ$ )
```

```
int newAngleM3 = actualAngleM3 + currentAngleM3;  
newAngleM3 = constrain(newAngleM3, M3_MIN, M3_MAX);  
if (newAngleM3 != actualAngleM3) {  
    long deltaM3 = (long)(newAngleM3 - actualAngleM3) * STEPS_PER_DEG_M3;  
    stepperM3.move(deltaM3);  
    actualAngleM3 = newAngleM3;  
}
```

```
// M4 (0-720°)
```

```
int newAngleM4 = actualAngleM4 + (currentAngleM4 - prevAngleM4);  
newAngleM4 = constrain(newAngleM4, M4_MIN, M4_MAX);
```



```
long deltaM4 = (long)(newAngleM4 - actualAngleM4) * STEPS_PER_DEG_M4;
```

```
if (deltaM4 != 0) {
```

```
    stepperM4.move(deltaM4);
```

```
    actualAngleM4 = newAngleM4;
```

```
}
```

```
// Solenoid (Momentary)
```

```
digitalWrite(RELAY_PIN, solenoidState ? LOW : HIGH);
```

```
Serial.print("M1="); Serial.print(deltaM1);
```

```
Serial.print(" M2="); Serial.print(deltaM2);
```

```
Serial.print(" M3="); Serial.print(actualAngleM3);
```

```
Serial.print(" M4="); Serial.print(actualAngleM4);
```

```
Serial.print(" SOL="); Serial.println(solenoidState ? "ON" : "OFF");
```

```
}
```

```
void loop() {
```

```
    while (SerialMega.available()) {
```

```
        String cmd = SerialMega.readStringUntil('\n');
```

```
        cmd.trim();
```

```
        if (cmd.length() > 0) parseCommand(cmd);
```

```
    }
```

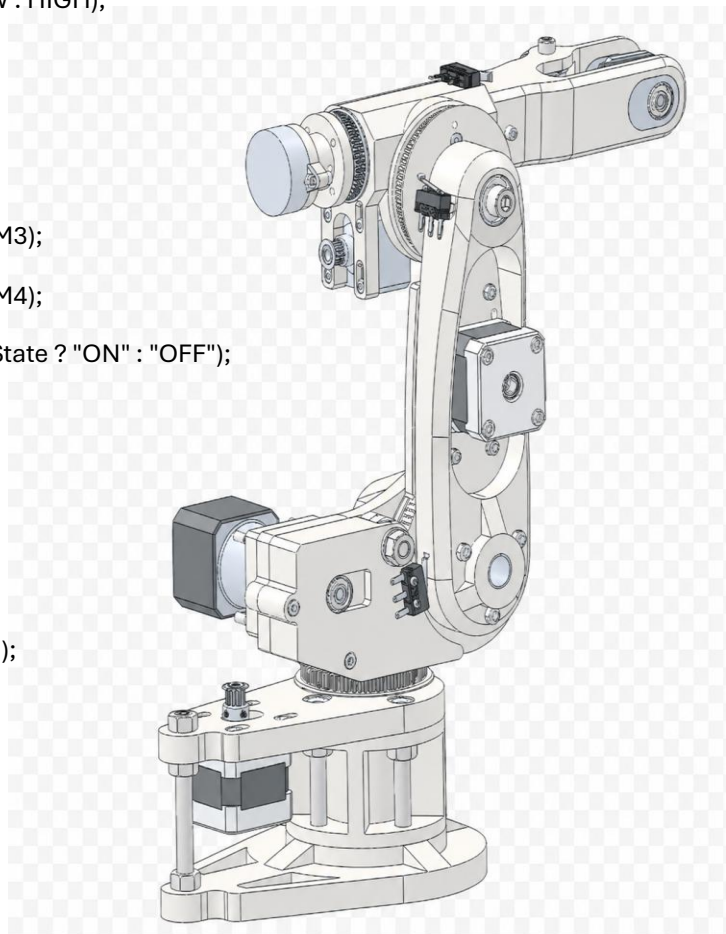
```
    stepperM1.run();
```

```
    stepperM2.run();
```

```
    stepperM3.run();
```

```
    stepperM4.run();
```

```
}
```



ESP32 SLAVE

```
#include <esp_now.h>
```

```
#include <WiFi.h>
```

```
uint8_t masterMAC[] = {0x44, 0x1D, 0x64, 0xFA, 0x2D, 0x94};
```

```
#define TX_PIN 17
```

```
#define RX_PIN 16
```

```
typedef struct {
```

```
    int16_t m1, m2, m3, m4;
```

```
    bool solenoidActive;
```

```
} RobotData;
```

```
RobotData receivedData;
```

```
volatile bool newData = false;
```

```
void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
```

```
    if (len == sizeof(RobotData)) {
```

```
        memcpy(&receivedData, data, sizeof(RobotData));
```

```
        newData = true;
```

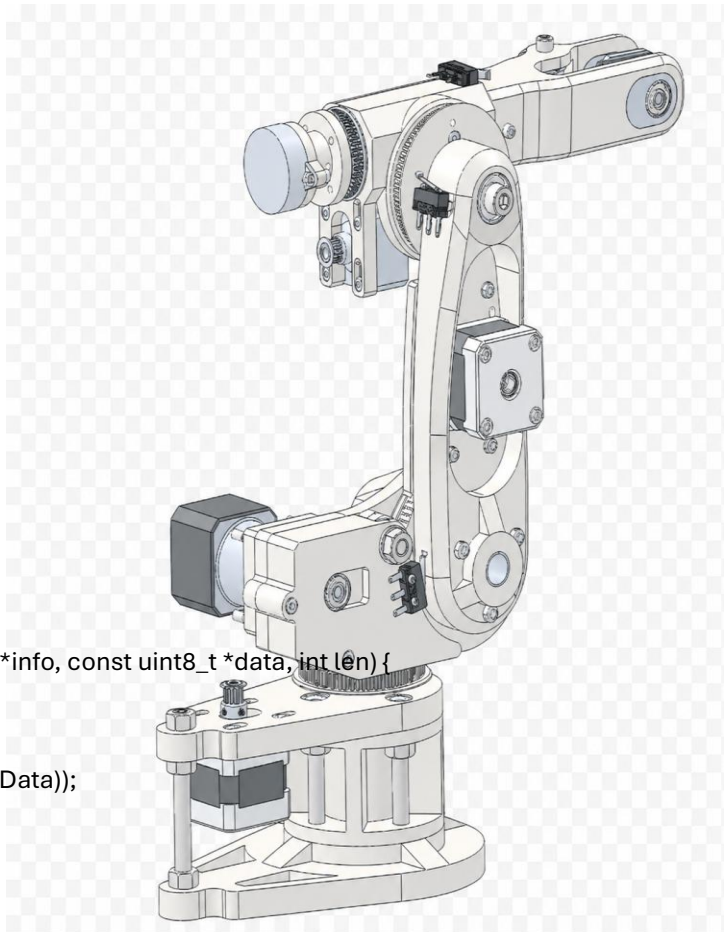
```
    }
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    Serial1.begin(115200, SERIAL_8N1, RX_PIN, TX_PIN);
```



```

WiFi.mode(WIFI_STA);

if (esp_now_init() != ESP_OK) { Serial.println("ESP-NOW Failed!"); return; }

esp_now_register_recv_cb(OnDataRecv);


esp_now_peer_info_t peerInfo = {};

memcpy(peerInfo.peer_addr, masterMAC, 6);

peerInfo.channel = 0; peerInfo.encrypt = false;

esp_now_add_peer(&peerInfo);

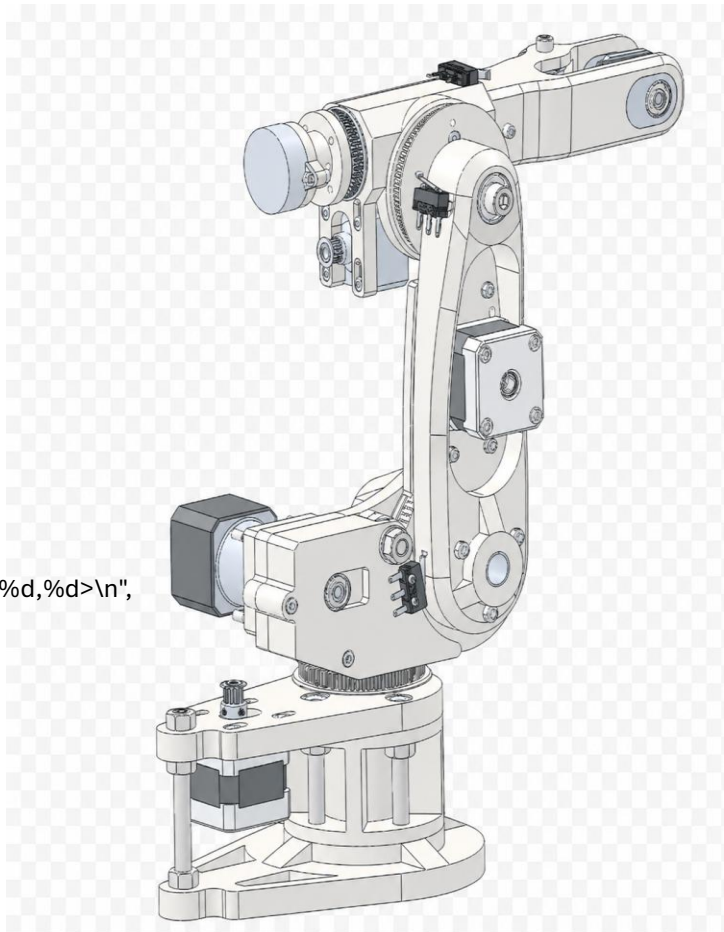

Serial.println("SLAVE Ready");
}

void loop() {
  if (newData) {
    newData = false;

    char buffer[64];
    snprintf(buffer, sizeof(buffer), "< %d,%d,%d,%d,%d>\n",
      receivedData.m1,
      receivedData.m2,
      receivedData.m3,
      receivedData.m4,
      receivedData.solenoidActive ? 1 : 0);

    Serial1.print(buffer);
    Serial.print("> Mega: ");
    Serial.print(buffer);
  }
  delay(10);}

```



ESP32 MASTER

/*

* ESP32 MASTER - Gyro M1,M2,M4 + Buttons M3 + Button Solenoid

* Gyro 1: M1, M2, M4

* Buttons: M3+ (GPIO25), M3- (GPIO26)

* Button: Solenoid (GPIO12) - momentary (press=ON, release=OFF)

*/

```
#include <Wire.h>
```

```
#include <esp_now.h>
```

```
#include <WiFi.h>
```

```
uint8_t slaveMAC[] = {0x44, 0x1D, 0x64, 0xF6, 0x3B, 0xC0};
```

```
// ===== MPU6050 =====
```

```
#define SDA_PIN 21
```

```
#define SCL_PIN 22
```

```
#define MPU_ADDR 0x68
```

```
// ===== BUTTONS =====
```

```
#define BTN_M3_PLUS 25
```

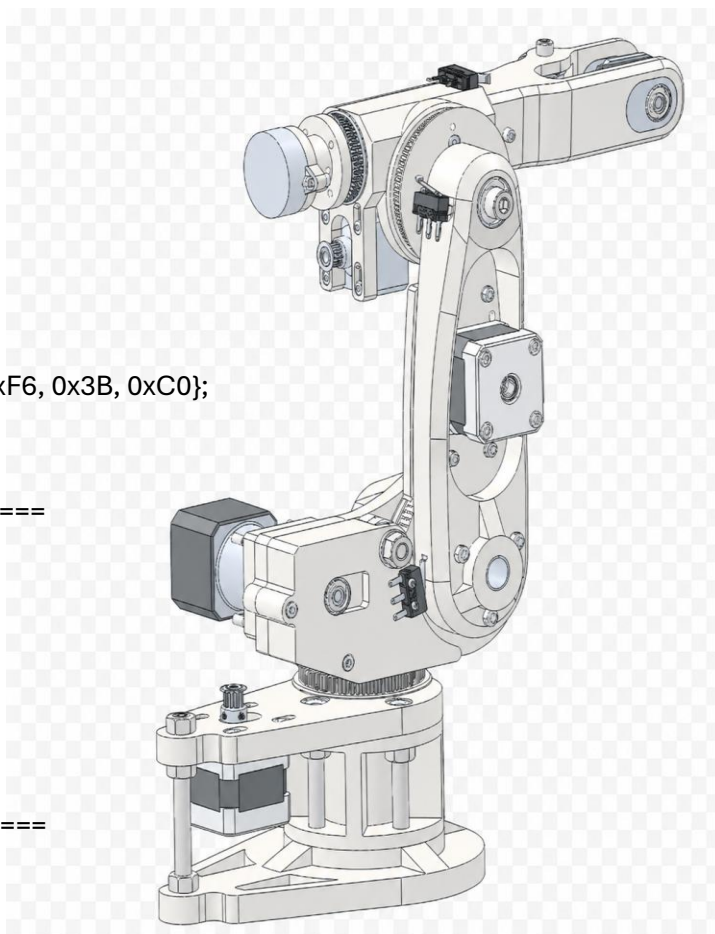
```
#define BTN_M3_MINUS 26
```

```
#define BTN_SOLENOID 12
```

```
// ===== LEDs =====
```

```
#define LED_M1_PLUS 32
```

```
#define LED_M1_MINUS 33
```



```
#define LED_M2_PLUS 19  
#define LED_M2_MINUS 18  
#define LED_M4_PLUS 5  
#define LED_M4_MINUS 17  
#define LED_M3_PLUS 16  
#define LED_M3_MINUS 4  
#define LED_SOLENOID 13  
#define LED_GYRO 27
```

```
// ===== DATA STRUCTURE =====
```

```
typedef struct {  
    int16_t m1;    // Gyro X  
    int16_t m2;    // Gyro Y  
    int16_t m3;    // Button: +1, -1, 0  
    int16_t m4;    // Gyro Z  
    bool solenoidActive; // Button: true while pressed  
} RobotData;
```

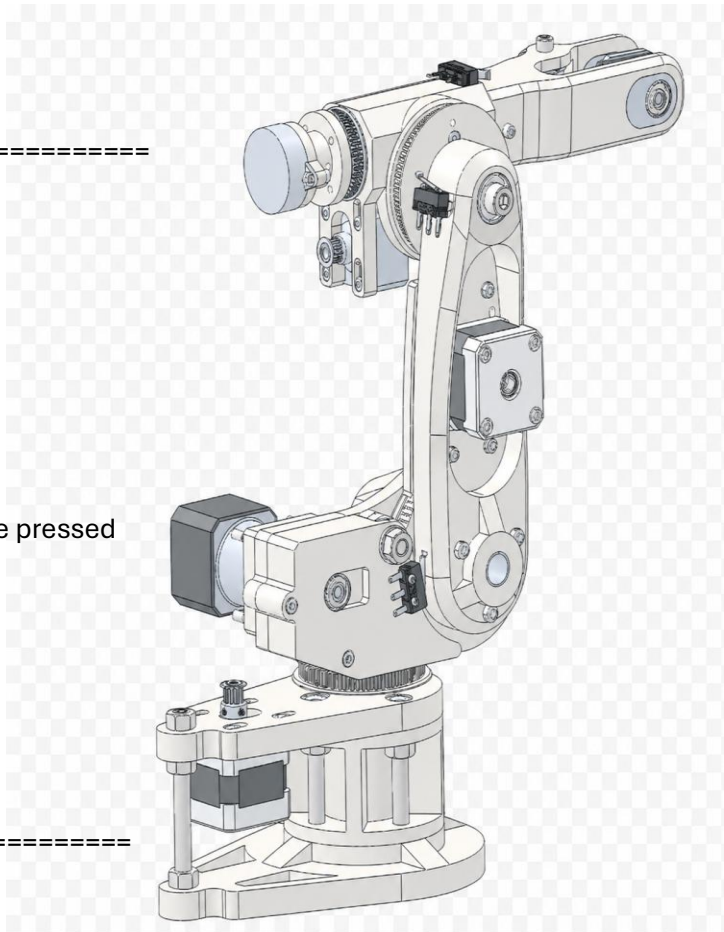
```
RobotData dataToSend;
```

```
// ===== MPU REGISTERS =====
```

```
#define MPU6050_PWR_MGMT_1 0x6B  
#define MPU6050_GYRO_XOUT_H 0x43
```

```
// ===== VARIABLES =====
```

```
float angleM1 = 0, angleM2 = 0, angleM4 = 0;  
unsigned long lastTime = 0;
```




```
// Button debounce
```

```
bool lastBtnM3Plus = HIGH, lastBtnM3Minus = HIGH, lastBtnSol = HIGH;
```

```
unsigned long debounceM3Plus = 0, debounceM3Minus = 0, debounceSol = 0;
```

```
#define DEBOUNCE_DELAY 30
```

```
// ===== ESP-NOW =====
```

```
void OnDataSent(const esp_now_send_info_t *info, esp_now_send_status_t status) {}
```

```
// ===== MPU FUNCTIONS =====
```

```
void initMPU() {
```

```
    Wire.beginTransmission(MPU_ADDR);
```

```
    Wire.write(MPU6050_PWR_MGMT_1);
```

```
    Wire.write(0x00);
```

```
    Wire.endTransmission(true);
```

```
    delay(10);
```

```
}
```

```
void readMPU(int16_t &gx, int16_t &gy, int16_t &gz) {
```

```
    Wire.beginTransmission(MPU_ADDR);
```

```
    Wire.write(MPU6050_GYRO_XOUT_H);
```

```
    if (Wire.endTransmission(false) != 0) { gx=gy=gz=0; return; }
```

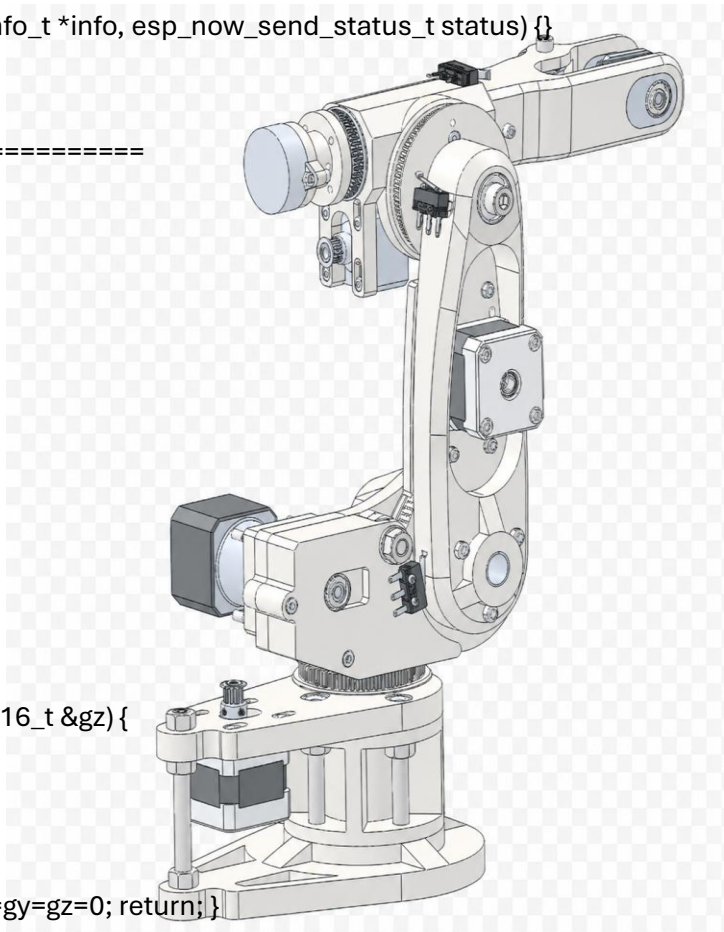
```
    if (Wire.requestFrom((uint8_t)MPU_ADDR, (uint8_t)6, (uint8_t)true) != 6) { gx=gy=gz=0; return; }
```

```
    gx = (Wire.read()<<8)|Wire.read();
```

```
    gy = (Wire.read()<<8)|Wire.read();
```

```
    gz = (Wire.read()<<8)|Wire.read();
```

```
}
```



```
// ===== SETUP =====
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  // I2C
```

```
  Wire.begin(SDA_PIN, SCL_PIN);
```

```
  Wire.setClock(100000);
```

```
  initMPU();
```

```
  // Buttons (INPUT_PULLUP = HIGH when not pressed, LOW when pressed)
```

```
  pinMode(BTN_M3_PLUS, INPUT_PULLUP);
```

```
  pinMode(BTN_M3_MINUS, INPUT_PULLUP);
```

```
  pinMode(BTN_SOLENOID, INPUT_PULLUP);
```

```
  // LEDs
```

```
  pinMode(LED_M1_PLUS, OUTPUT); pinMode(LED_M1_MINUS, OUTPUT);
```

```
  pinMode(LED_M2_PLUS, OUTPUT); pinMode(LED_M2_MINUS, OUTPUT);
```

```
  pinMode(LED_M4_PLUS, OUTPUT); pinMode(LED_M4_MINUS, OUTPUT);
```

```
  pinMode(LED_M3_PLUS, OUTPUT); pinMode(LED_M3_MINUS, OUTPUT);
```

```
  pinMode(LED_SOLENOID, OUTPUT);
```

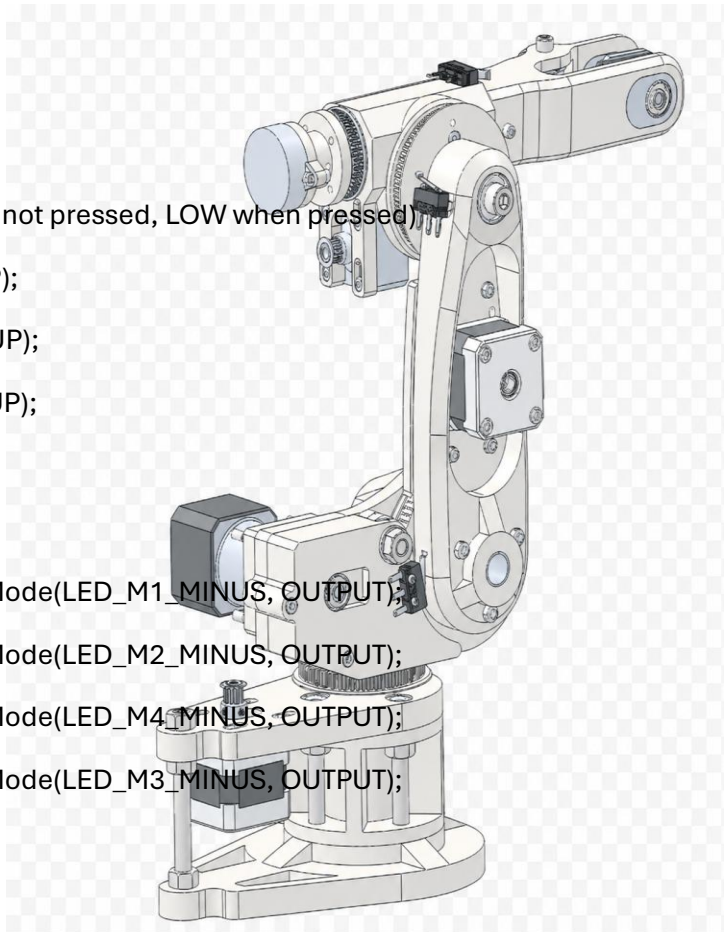
```
  pinMode(LED_GYRO, OUTPUT);
```

```
  // All LEDs off
```

```
  digitalWrite(LED_M1_PLUS, LOW); digitalWrite(LED_M1_MINUS, LOW);
```

```
  digitalWrite(LED_M2_PLUS, LOW); digitalWrite(LED_M2_MINUS, LOW);
```

```
  digitalWrite(LED_M4_PLUS, LOW); digitalWrite(LED_M4_MINUS, LOW);
```



```
digitalWrite(LED_M3_PLUS, LOW); digitalWrite(LED_M3_MINUS, LOW);  
  
digitalWrite(LED_SOLENOID, LOW);  
  
digitalWrite(LED_GYRO, LOW);  
  
// Init data  
  
dataToSend.m1 = 0; dataToSend.m2 = 0; dataToSend.m3 = 0; dataToSend.m4 = 0;  
  
dataToSend.solenoidActive = false;
```

```
// ESP-NOW
```

```
WiFi.mode(WIFI_STA);
```

```
if (esp_now_init() != ESP_OK) { Serial.println("ESP-NOW Failed!"); return; }
```

```
esp_now_register_send_cb(OnDataSent);
```

```
esp_now_peer_info_t peerInfo = {};
```

```
memcpy(peerInfo.peer_addr, slaveMAC, 6);
```

```
peerInfo.channel = 0; peerInfo.encrypt = false;
```

```
esp_now_add_peer(&peerInfo);
```

```
Serial.println("MASTER Ready");
```

```
Serial.println("Gyro: M1,M2,M4 | Buttons: M3+/- | Momentary: SOL");
```

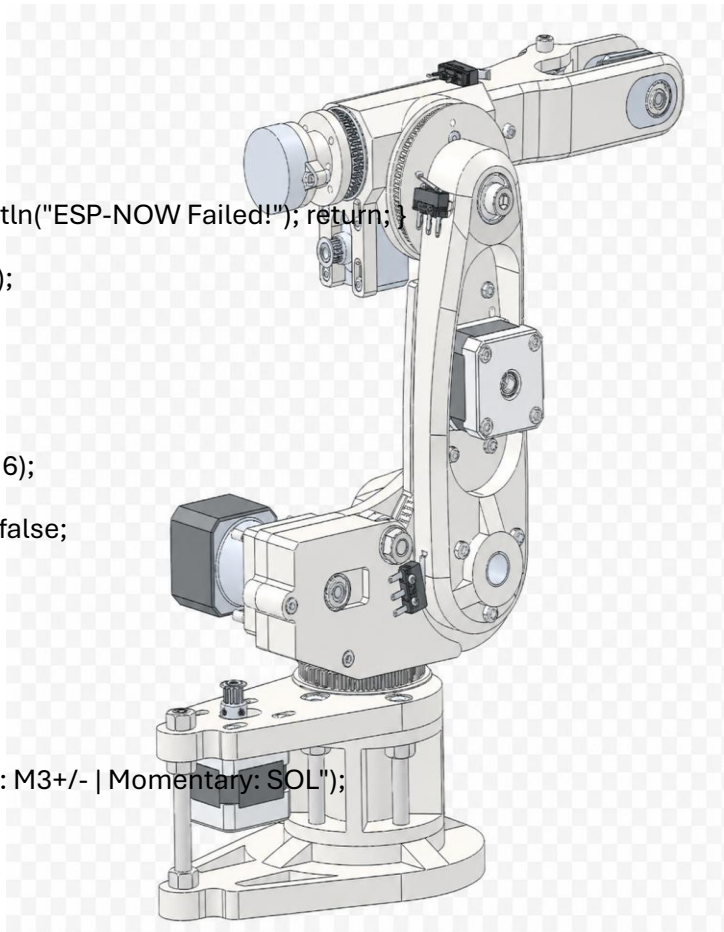
```
}
```

```
// ===== READ BUTTONS =====
```

```
void readButtons() {
```

```
    // Reset M3
```

```
    dataToSend.m3 = 0;
```



```

// M3+ (RIGHT)

bool reading = digitalRead(BTN_M3_PLUS);

if (reading != lastBtnM3Plus) debounceM3Plus = millis();

if ((millis() - debounceM3Plus) > DEBOUNCE_DELAY) {

    if (reading == LOW) dataToSend.m3 = 1; // Pressed

}

lastBtnM3Plus = reading;

```

```

// M3- (LEFT)

reading = digitalRead(BTN_M3_MINUS);

if (reading != lastBtnM3Minus) debounceM3Minus = millis();

if ((millis() - debounceM3Minus) > DEBOUNCE_DELAY) {

    if (reading == LOW) dataToSend.m3 = -1; // Pressed

}

lastBtnM3Minus = reading;

```

```

// Solenoid (MOMENTARY - while pressed)

reading = digitalRead(BTN_SOLENOID);

if (reading != lastBtnSol) debounceSol = millis();

if ((millis() - debounceSol) > DEBOUNCE_DELAY) {

    dataToSend.solenoidActive = (reading == LOW); // LOW = pressed

}

lastBtnSol = reading;

}

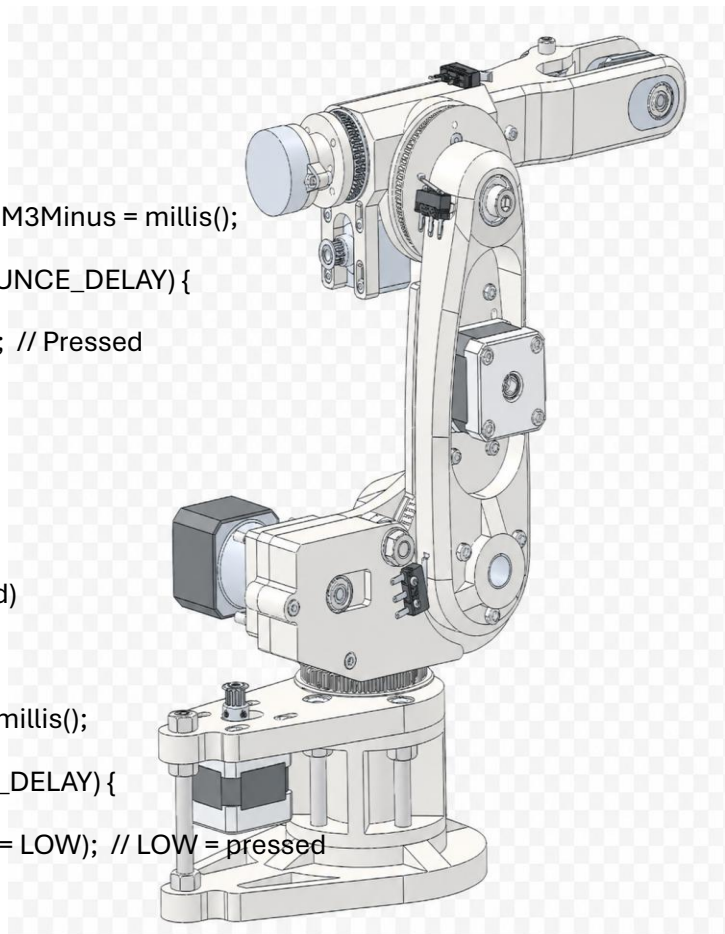
```

```

// ===== UPDATE LEDs =====

void updateLEDs() {

```



```

digitalWrite(LED_M1_PLUS, (dataToSend.m1 > 3) ? HIGH : LOW);
digitalWrite(LED_M1_MINUS, (dataToSend.m1 < -3) ? HIGH : LOW);
digitalWrite(LED_M2_PLUS, (dataToSend.m2 > 3) ? HIGH : LOW);
digitalWrite(LED_M2_MINUS, (dataToSend.m2 < -3) ? HIGH : LOW);
digitalWrite(LED_M4_PLUS, (dataToSend.m4 > 3) ? HIGH : LOW);
digitalWrite(LED_M4_MINUS, (dataToSend.m4 < -3) ? HIGH : LOW);
digitalWrite(LED_M3_PLUS, (dataToSend.m3 > 0) ? HIGH : LOW);
digitalWrite(LED_M3_MINUS, (dataToSend.m3 < 0) ? HIGH : LOW);
digitalWrite(LED_SOLENOID, dataToSend.solenoidActive ? HIGH : LOW);
digitalWrite(LED_GYRO, HIGH);
}

// ===== LOOP =====

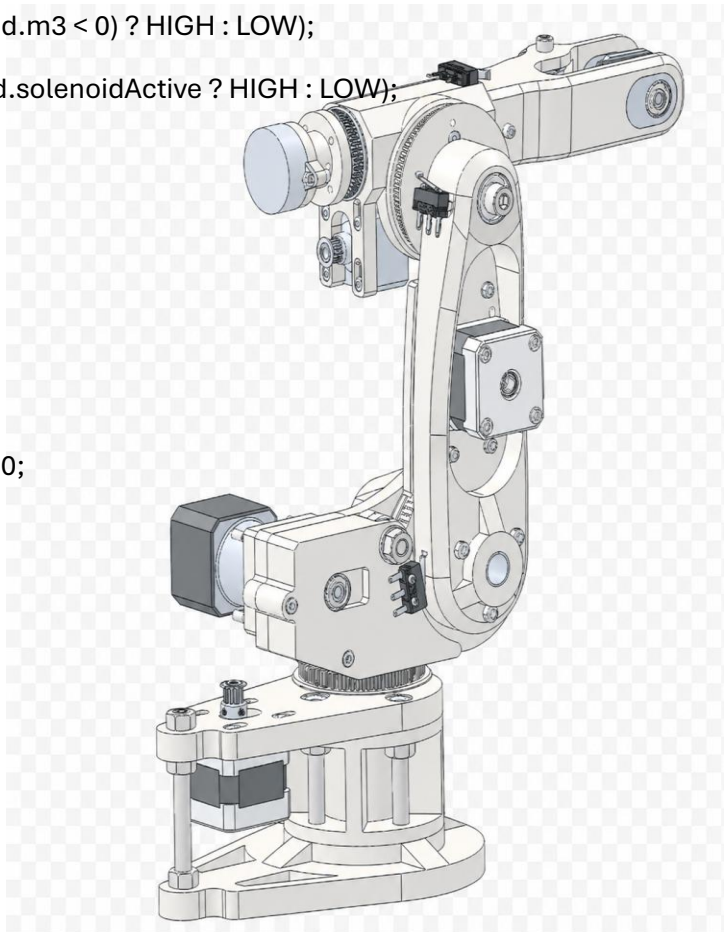
void loop() {
    unsigned long currentTime = millis();
    float dt = (currentTime - lastTime) / 1000.0;
    if (dt <= 0 || dt > 1.0) dt = 0.02;
    lastTime = currentTime;

    // Read Gyro
    int16_t gx, gy, gz;
    readMPU(gx, gy, gz);

    float gx_dps = gx / 131.0;
    float gy_dps = gy / 131.0;
    float gz_dps = gz / 131.0;

    // Integrate

```




```

angleM1 += gx_dps * dt; // M1
angleM2 += gy_dps * dt; // M2
angleM4 += gz_dps * dt; // M4

// M4 limits 0-720
if (angleM4 < 0) angleM4 = 0;
if (angleM4 > 720) angleM4 = 720;

// Read buttons
readButtons();

// Prepare data
dataToSend.m1 = (int16_t)angleM1;
dataToSend.m2 = (int16_t)angleM2;
dataToSend.m4 = (int16_t)angleM4;

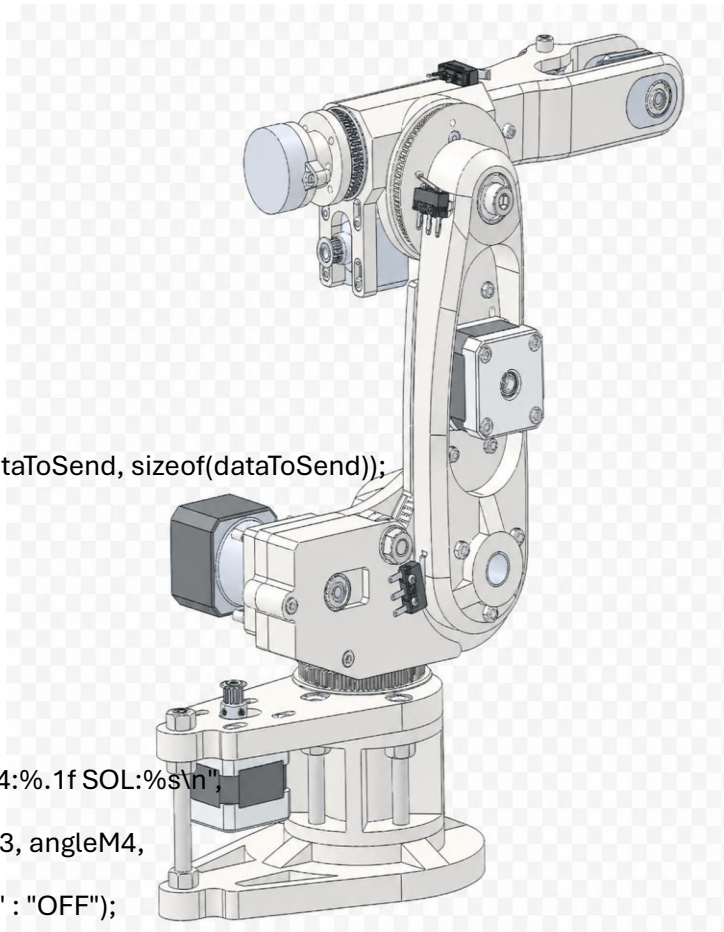
// Send
esp_now_send(slaveMAC, (uint8_t *)&dataToSend, sizeof(dataToSend));

// LEDs
updateLEDs();

// Debug
Serial.printf("M1:%.1f M2:%.1f M3:%d M4:%.1f SOL:%s\n",
              angleM1, angleM2, dataToSend.m3, angleM4,
              dataToSend.solenoidActive ? "ON" : "OFF");

delay(50);}

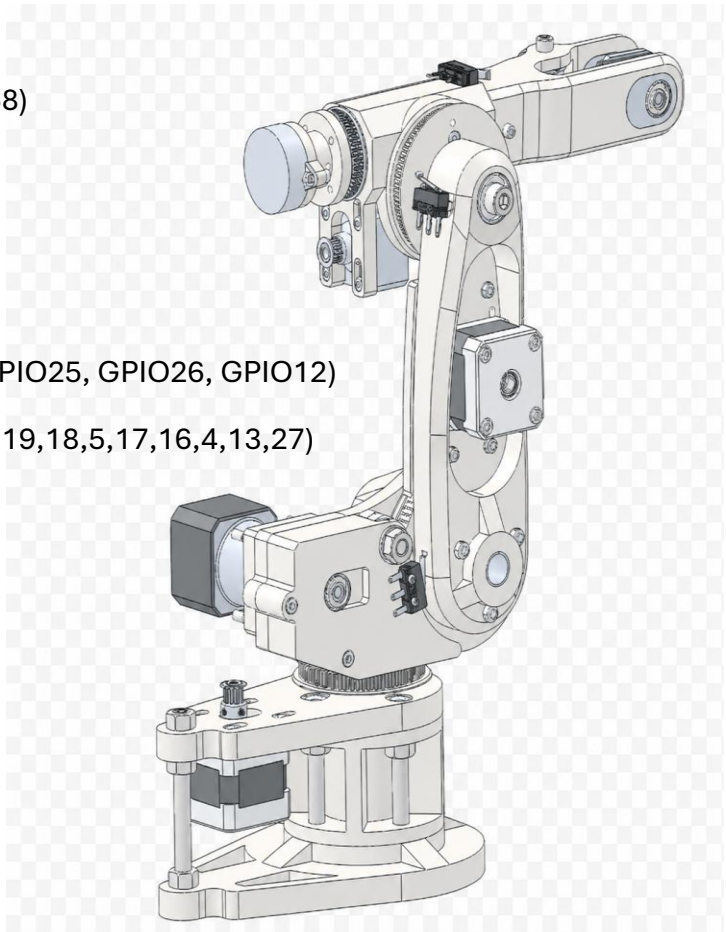
```



ESP32 MASTER Algorithm

START

```
|
|
|─► Initialize Serial (115200)
|
|
|─► Initialize I2C (SDA=21, SCL=22)
|
|
|─► Initialize MPU6050 (Address 0x68)
|  └─► Wake up (Write 0x00 to 0x6B)
|
|
|─► Initialize Pins
|  └─► 3 Buttons: INPUT_PULLUP (GPIO25, GPIO26, GPIO12)
|    └─► 10 LEDs: OUTPUT (GPIO32,33,19,18,5,17,16,4,13,27)
|      └─► All LEDs = OFF
|
|
|─► Initialize ESP-NOW
|  └─► WiFi.mode(WIFI_STA)
|  └─► esp_now_init()
|  └─► Register Send Callback
|    └─► Add Peer (Slave MAC)
|
|
|─► Initialize Data Structure
|  └─► m1=0, m2=0, m3=0, m4=0, solenoidActive=false
|
|
|─► MAIN LOOP
```



|
└─► Calculate dt (delta time)

|
└─► READ MPU6050

| └─► Read GX, GY, GZ (Registers 0x43-0x48)

| └─► Convert to deg/s (Divide by 131)

| └─► Integrate Angles

| └─► angleM1 += GX * dt

| └─► angleM2 += GY * dt

| └─► angleM4 += GZ * dt

| └─► Constrain: $0 \leq \text{angleM4} \leq 720$

|
└─► READ BUTTONS (with Debounce 30ms)

| └─► M3+ Button (GPIO25)

| └─► IF pressed → m3 = +1

| └─► M3- Button (GPIO26)

| └─► IF pressed → m3 = -1

| └─► SOLENOID Button (GPIO12)

| └─► IF pressed → solenoidActive = true

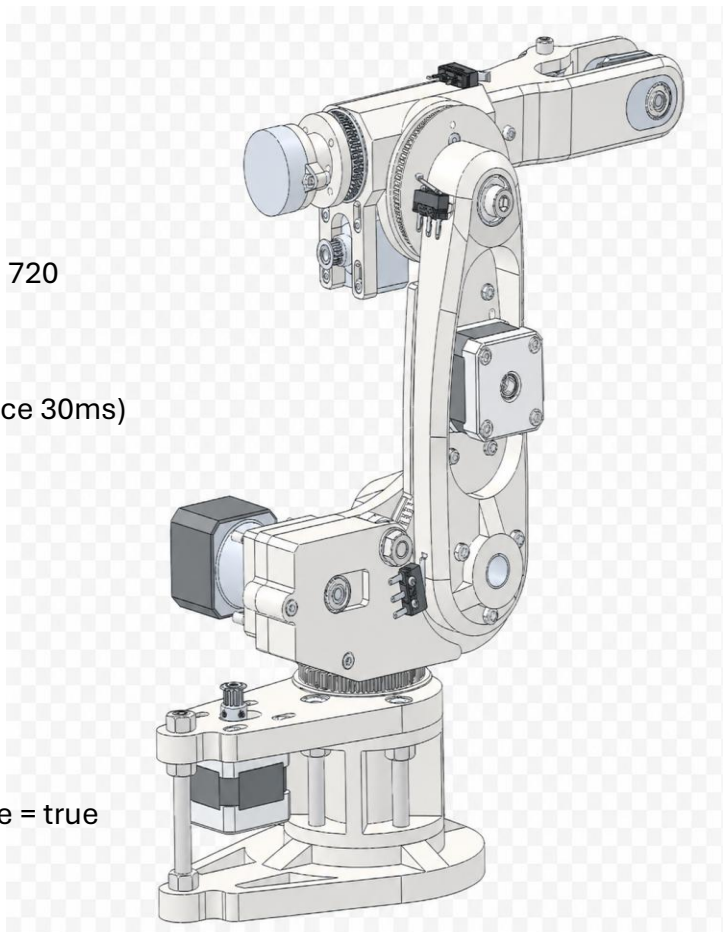
| ELSE → solenoidActive = false

|
└─► PREPARE DATA PACKET

| └─► data.m1 = angleM1

| └─► data.m2 = angleM2

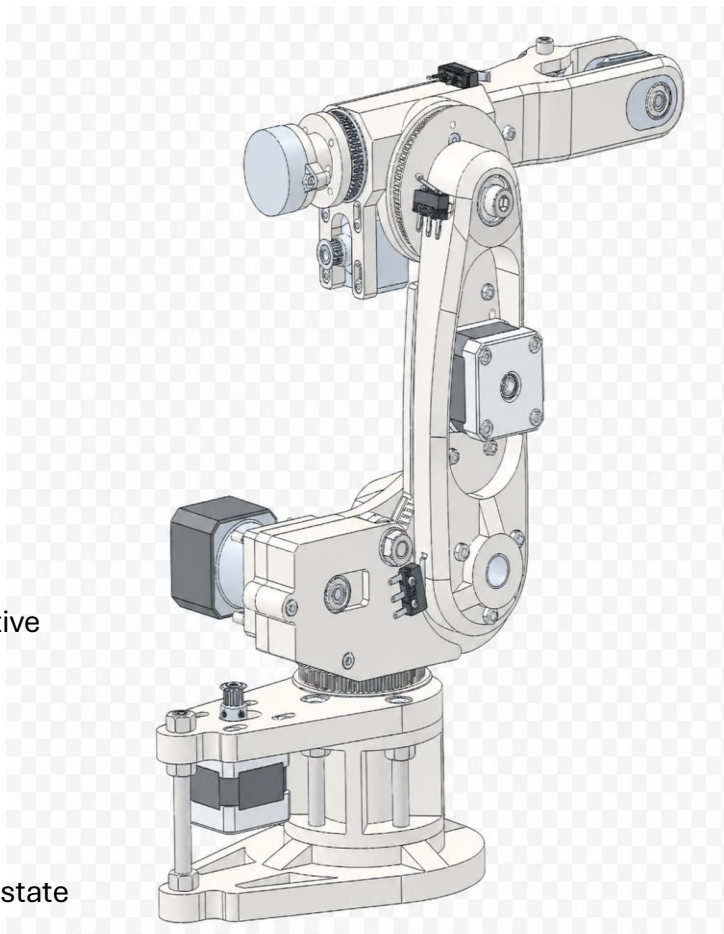
| └─► data.m3 = buttonM3 (+1, -1, or 0)



```

|   |--> data.m4 = angleM4
|   |--> data.solenoidActive = buttonState
|
|--> SEND via ESP-NOW
|   |--> esp_now_send(slaveMAC, &data, sizeof(data))
|
|--> UPDATE LEDs
|   |--> M1+ LED: ON if m1 > 3
|   |--> M1- LED: ON if m1 < -3
|   |--> M2+ LED: ON if m2 > 3
|   |--> M2- LED: ON if m2 < -3
|   |--> M4+ LED: ON if m4 > 3
|   |--> M4- LED: ON if m4 < -3
|   |--> M3+ LED: ON if m3 > 0
|   |--> M3- LED: ON if m3 < 0
|   |--> SOL LED: ON if solenoidActive
|   |--> GYRO LED: Always ON
|
|--> PRINT DEBUG INFO
|   |--> Serial: M1, M2, M3, M4, SOL state
|
|--> DELAY 50ms
|   |--> LOOP BACK

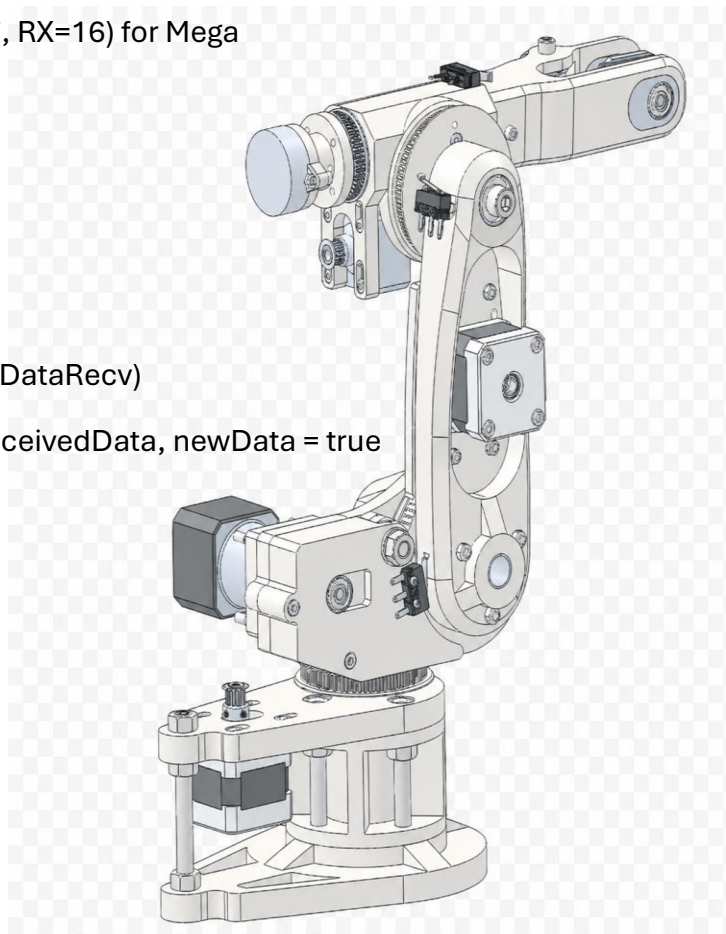
```



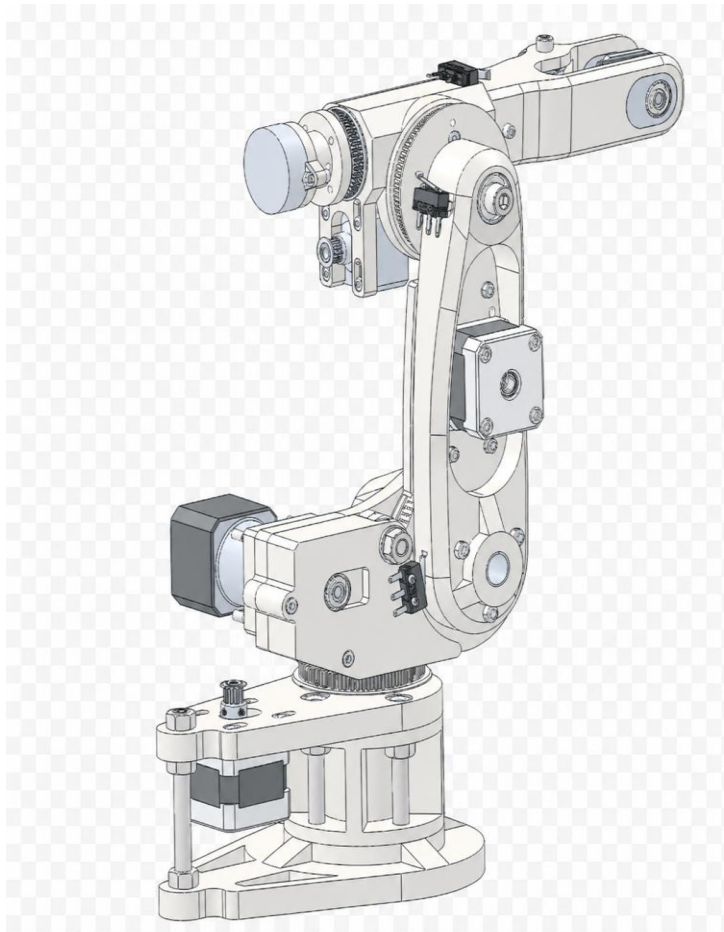
ESP32 SLAVE Algorithm

START

```
|
|
|--> Initialize Serial (115200) for Debug
|
|--> Initialize Serial1 (115200, TX=17, RX=16) for Mega
|
|--> Initialize ESP-NOW
|   |--> WiFi.mode(WIFI_STA)
|   |--> esp_now_init()
|   |--> Register Receive Callback (OnDataRecv)
|       |--> IF data received → Copy to receivedData, newData = true
|
|--> Add Peer (Master MAC)
|
|--> MAIN LOOP
|
|   |--> CHECK newData flag
|   |
|   |   |--> IF newData == true
|   |   |
|   |   |   |--> Clear flag: newData = false
|   |   |
|   |   |--> FORMAT UART STRING
```




```
| | L▶ "<m1,m2,m3,m4,solenoid>\n"  
| | Example: "<45,-30,1,180,1>\n"  
| |  
| |└─▶ SEND to Arduino Mega via Serial1  
| | L▶ Serial1.print(buffer)  
| |  
| | L▶ PRINT DEBUG  
| | L▶ Serial: "→ Mega: <data>"  
|  
L▶ DELAY 10ms  
L▶ LOOP BACK
```



Arduino Mega Algorithm

START

|

|→► Initialize Serial (115200) for Debug

|

|→► Initialize Serial1 (115200) for ESP32 Slave

|

|→► Initialize Relay Pin (31)

| └─► OUTPUT, HIGH (Active LOW = OFF)

|

|→► Initialize Stepper Motors

| |→► M1: Pin32(STEP), Pin34(DIR)

| |→► M2: Pin41(STEP), Pin43(DIR)

| |→► M3: Pin36(STEP), Pin46(DIR)

| └─► M4: Pin20(PUL), Pin21(DIR)

| └─► Set Speed & Acceleration for all

|

|→► Set Home Position (0,0,0,0)

|

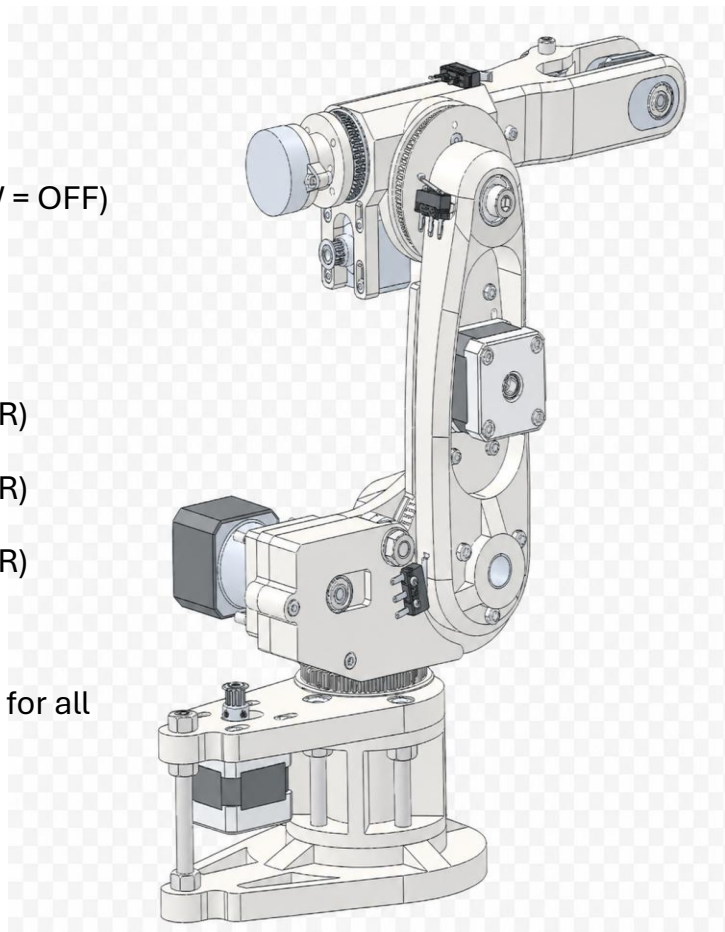
|→► MAIN LOOP

|

| |→► CHECK Serial1 Available

|

| └─► IF data available



└─► READ String until '\n'

└─► PARSE COMMAND

└─► Format: "<m1,m2,m3,m4,solenoid>"

Example: "<45,-30,1,180,1>"

└─► EXTRACT VALUES

└─► m1 = values[0]

└─► m2 = values[1]

└─► m3 = values[2] (+1 or -1)

└─► m4 = values[3]

└─► solenoid = values[4] (0 or 1)

└─► CALCULATE DELTA (Difference from previous)

└─► $\text{deltaM1} = (\text{m1} - \text{prevM1}) * \text{STEPS_PER_DEG}$

└─► $\text{deltaM2} = (\text{m2} - \text{prevM2}) * \text{STEPS_PER_DEG}$

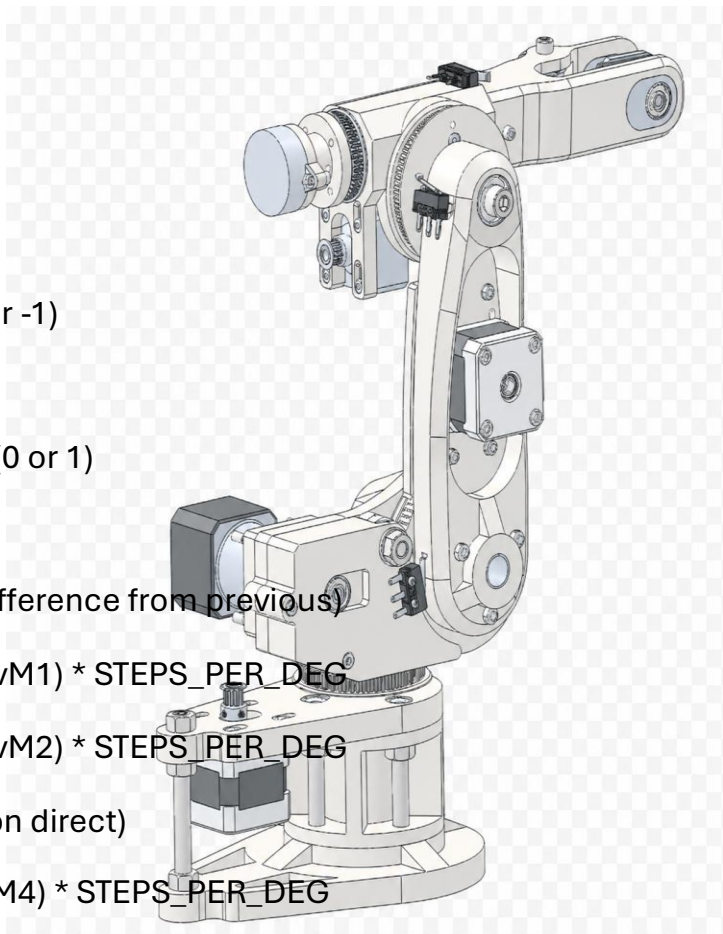
└─► $\text{deltaM3} = \text{m3}$ (button direct)

└─► $\text{deltaM4} = (\text{m4} - \text{prevM4}) * \text{STEPS_PER_DEG}$

└─► MOVE M1

└─► `stepperM1.move(deltaM1)`

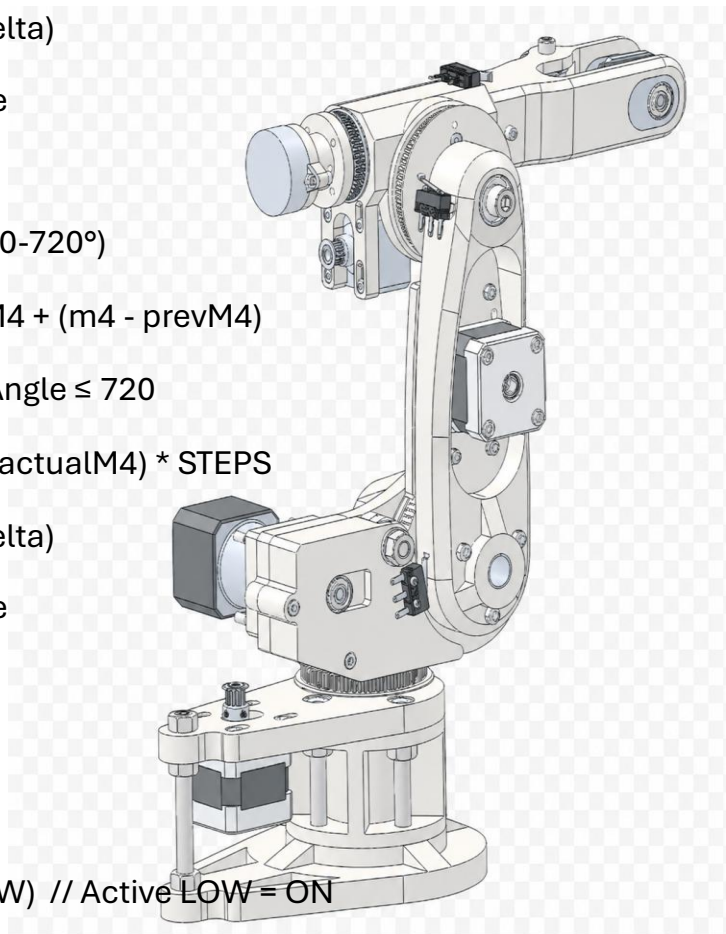
└─► MOVE M2



```

|   |   └─▶ stepperM2.move(deltaM2)
|   |
|   └─▶ MOVE M3 (with Limits ±90°)
|   |   └─▶ newAngle = actualM3 + m3
|   |   └─▶ Constrain:  $-90 \leq \text{newAngle} \leq 90$ 
|   |   └─▶  $\text{delta} = (\text{newAngle} - \text{actualM3}) * \text{STEPS}$ 
|   |   └─▶ stepperM3.move(delta)
|   |   └─▶ actualM3 = newAngle
|   |
|   └─▶ MOVE M4 (with Limits 0-720°)
|   |   └─▶  $\text{newAngle} = \text{actualM4} + (\text{m4} - \text{prevM4})$ 
|   |   └─▶ Constrain:  $0 \leq \text{newAngle} \leq 720$ 
|   |   └─▶  $\text{delta} = (\text{newAngle} - \text{actualM4}) * \text{STEPS}$ 
|   |   └─▶ stepperM4.move(delta)
|   |   └─▶ actualM4 = newAngle
|   |
|   └─▶ CONTROL SOLENOID
|   |   └─▶ IF solenoid == 1
|   |       └─▶ digitalWrite(31, LOW) // Active LOW = ON
|   |   ELSE
|   |       └─▶ digitalWrite(31, HIGH) // Active LOW = OFF
|   |
|   └─▶ UPDATE previous values
|   |   └─▶ prevM1=m1, prevM2=m2, prevM3=m3, prevM4=m4

```



```
| |
| └─▶ PRINT DEBUG
|   └─▶ Serial: M1, M2, M3, M4, SOL state
|
|─▶ RUN ALL STEPPERS (Non-blocking)
|   └─▶ stepperM1.run()
|   └─▶ stepperM2.run()
|   └─▶ stepperM3.run()
|   └─▶ stepperM4.run()
|
|─▶ LOOP BACK
```

